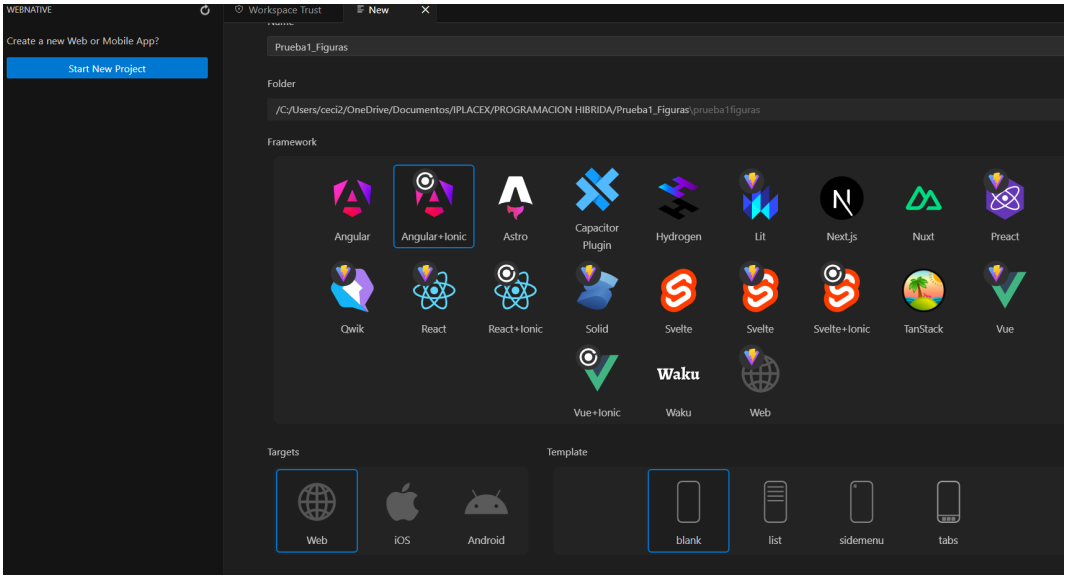
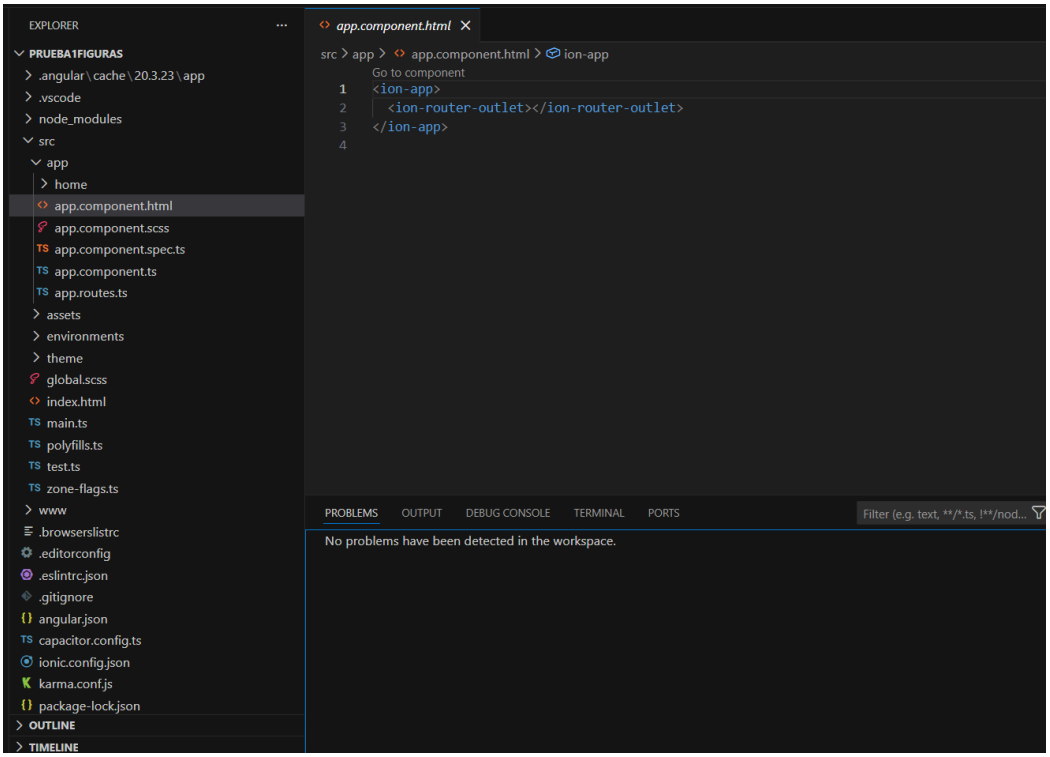
Institución: IPLACEX
Asignatura: Programación Híbrida
Profesor: Karen Vanessa Pizarro Osses
Estudiante: Cecilia Andreina Sanchez De Vicente
GitHub link:
https://github.com/ceciliaasanchezdv/B1-2026_PROGRAMACION-HIBRIDA.git

Creación del proyecto de Ionic + Angular para plataforma Web

En Visual Studio Code, cree el proyecto utilizando la extensión de Web Native:

- **Framework:** seleccioné **Angular + Ionic**
- **Target:** seleccioné **Web**
- **Template:** seleccioné **Blank**

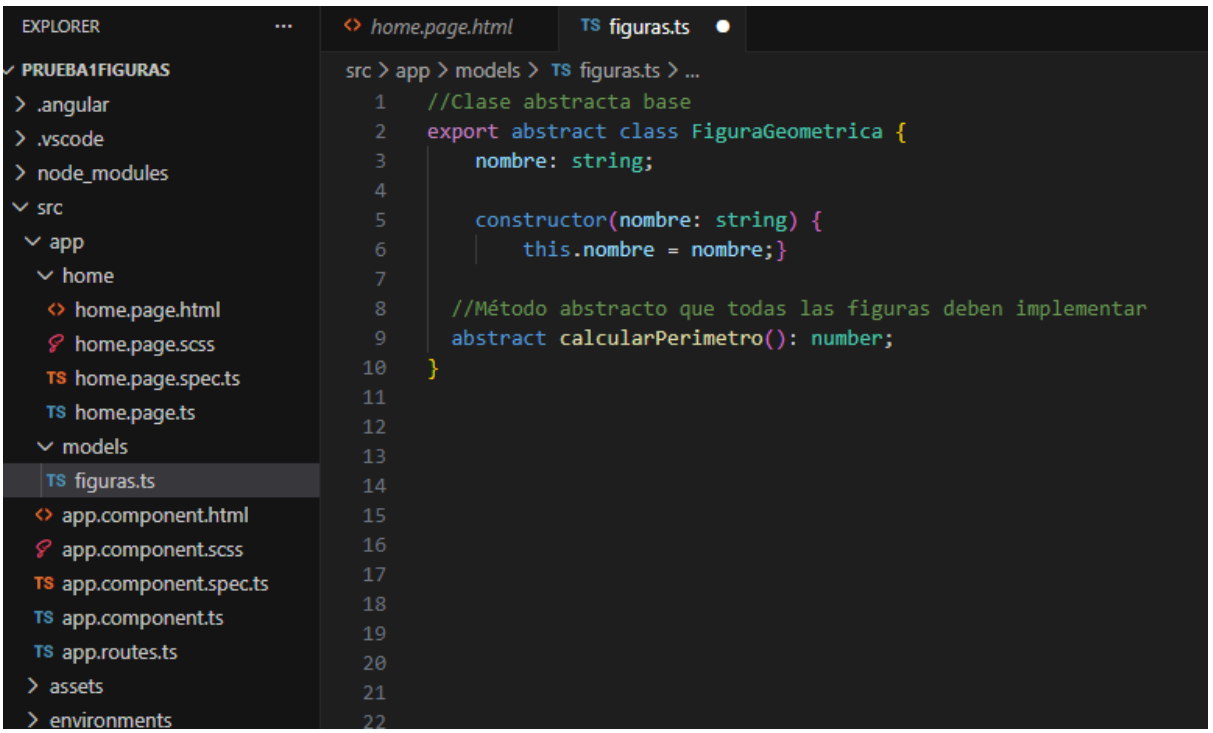




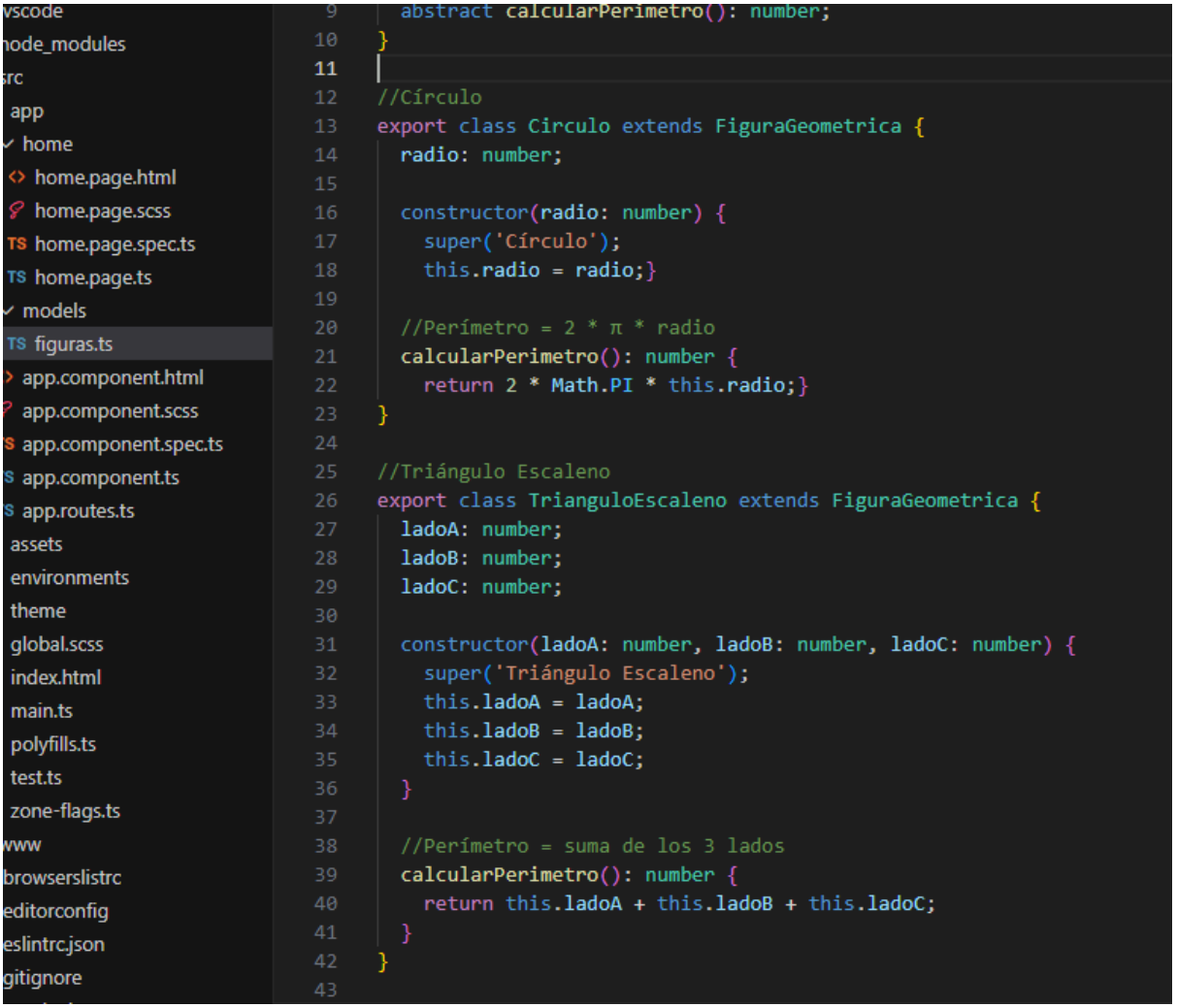
Programación de jerarquía de clases en Typescript (TS)

Cree un archivo typescript llamado figuras dentro de una nueva carpeta “models” en src/app. En este archivo se programaron las cuatro clases del diagrama UML.

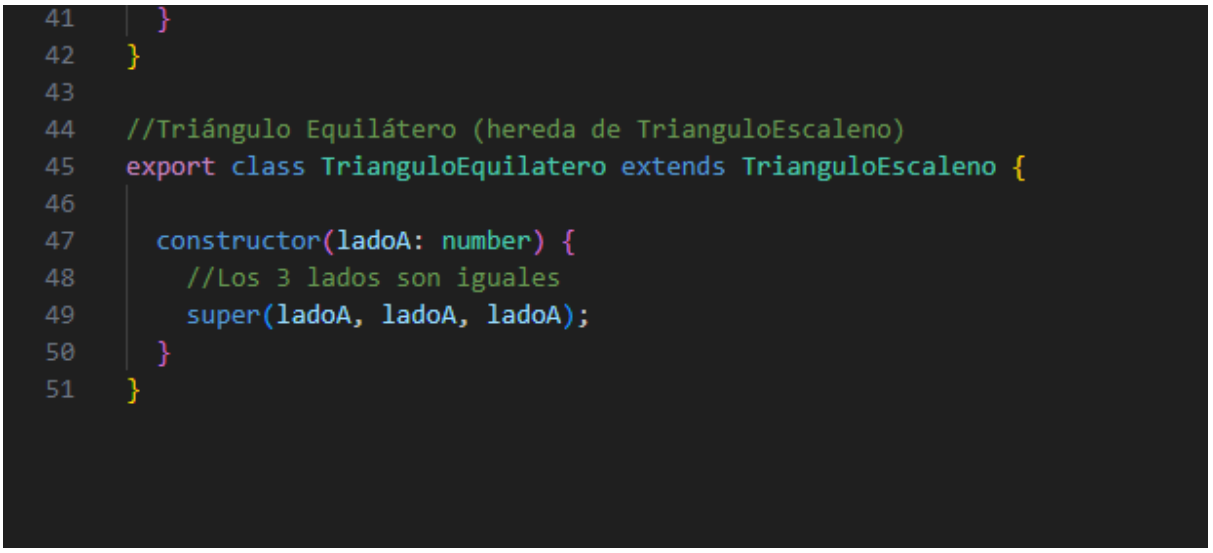
La primera clase es FiguraGeometrica, que se declaró como abstracta. Esto significa que no se puede usar directamente para crear objetos, sino que sirve como base para las demás clases. Tiene un atributo nombre y un método abstracto llamado calcularPerimetro() para su implementación en cada clase hija y definirlo a su manera.



Las clases Circulo y TrianguloEscaleno heredan de FiguraGeometrica usando extends. Cada una recibe sus propios datos en el constructor, por ejemplo el radio en el caso del círculo, y los tres lados en el caso del triángulo escaleno. En ambos constructores se llama a super() para inicializar el nombre de la figura, que viene de la clase padre. Cada una implementa calcularPerimetro() con su fórmula correspondiente: 2 * Math.PI * radio para el círculo, y la suma de los tres lados para el triángulo escaleno.

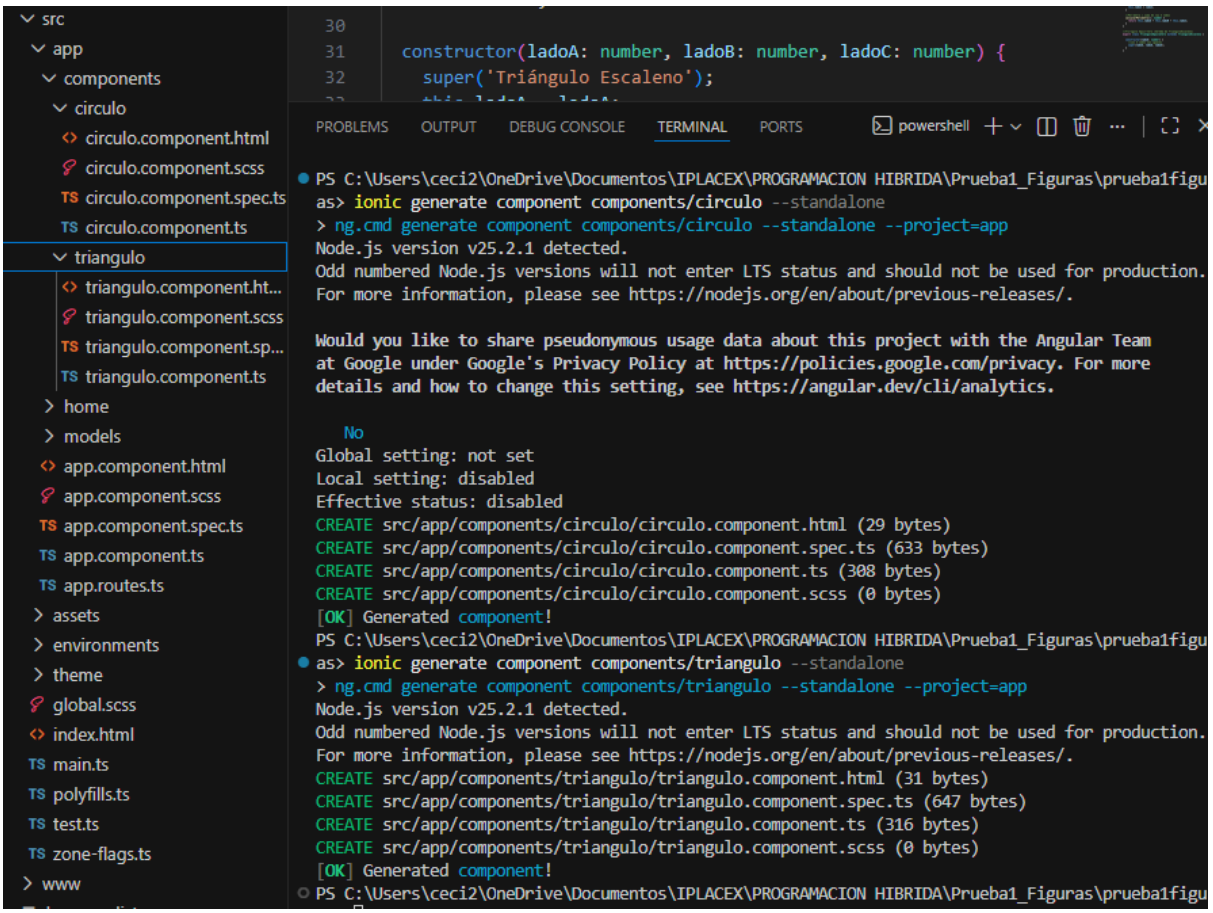


Por último, TrianguloEquilatero hereda de TrianguloEscaleno y no de FiguraGeometrica directamente, tal como indica el diagrama UML. Como un triángulo equilátero tiene los tres lados iguales, su constructor solo recibe un valor y lo pasa tres veces al constructor del padre mediante super(ladoA, ladoA, ladoA).



Creación de Componentes Círculo y Triángulo

Para crear los componentes utilice el comando **ionic generate component** desde la terminal, lo que generó automáticamente las carpetas circulo y triangulo dentro de src/app/components/, cada una con su archivo HTML y su archivo TypeScript. Agregue en el comando --standalone para que sea standalone predeterminadamente al crear el componente.



a. Desarrollo de la plantilla

La plantilla de ambos componentes se construyó usando componentes de Ionic. Se usó ion-card como contenedor principal para mostrar la información de cada figura de forma ordenada. Dentro de esta se agregó ion-img para mostrar una imagen de la figura (insertando imágenes guardadas en la carpeta assets/ del proyecto), campos de entrada con ion-input para que el usuario ingrese los valores necesarios, y un ion-button para desencadenar el cálculo al hacer click.

Para vincular los campos con las variables de la clase se usó [(ngModel)], que mantiene sincronizado lo que el usuario escribe con la variable correspondiente. El resultado se muestra usando la directiva *ngIf, que lo hace visible solo cuando ya existe un valor calculado.

HTML del Círculo

circulo.component.html × | circulo.component.scss | triangulo.component.html | home.page.html

src > app > components > circulo > circulo.component.html > ion-card > ion-card-header

Go to component

```html
<ion-card>
  <ion-card-header>
  <ion-card-subtitle>Perímetro</ion-card-subtitle>
  <ion-card-title>Círculo</ion-card-title>
</ion-card-header>

  <ion-card-content>
    <p>El perímetro de un círculo es la longitud de la línea que forma su borde.</p>
    <ion-img src="assets/circulo.png"></ion-img>

    <ion-item>
      <ion-label position="floating">Radio (cm)</ion-label>
      <ion-input
        type="number"
        [(ngModel)]="radio">
      </ion-input>
    </ion-item>

    <ion-button expand="block" (click)="calcular()">
      Calcular perímetro
    </ion-button>

    <p class="resultado" *ngIf="resultado !== null">
      El perímetro es {{ resultado | number:'1.2-2' }} cm
    </p>
  </ion-card-content>
</ion-card>
```

HTML del Triángulo

circulo.component.html | triangulo.component.html ●

src > app > components > triangulo > triangulo.component.html > ion-card > ion-card-content

Go to component

```html
<ion-card>
  <ion-card-header>
    <ion-card-title>Triángulo</ion-card-title>
    <ion-card-subtitle>Perímetro</ion-card-subtitle>
  </ion-card-header>

  <ion-card-content>
    <p>El perímetro de un triángulo es la suma de las longitudes
    de sus tres lados.</p>
    <ion-img src="assets/triangulo.png"></ion-img>

    <ion-item>
      <ion-label position="floating">Lado A (cm)</ion-label>
      <ion-input type="number" [(ngModel)]="ladoA"></ion-input>
    </ion-item>

    <ion-item>
      <ion-label position="floating">Lado B (cm)</ion-label>
      <ion-input type="number" [(ngModel)]="ladoB"></ion-input>
    </ion-item>

    <ion-item>
      <ion-label position="floating">Lado C (cm)</ion-label>
      <ion-input
      type="number"
      [(ngModel)]="ladoC"></ion-input>
    </ion-item>

    <ion-button expand="block" (click)="calcular()">
      Calcular el Perímetro
    </ion-button>

    <p *ngIf="resultado !== null">
      El perímetro es {{ resultado }} cm
    </p>

  </ion-card-content>
</ion-card>
```

b. Desarrollo de la clase en TypeScript

En la clase de cada componente se declararon las variables necesarias para almacenar los datos ingresados y el resultado.

Al hacer click, el método calcular() crea una instancia de la clase correspondiente (Circulo o TrianguloEscaleno) con los valores ingresados por el usuario, llama al método calcularPerimetro() y guarda el resultado en la variable, lo que hace que automáticamente aparezca en pantalla.

En el componente de Círculo se declararon dos variables: radio para almacenar el valor ingresado por el usuario, y resultado que parte en null para que el párrafo con el resultado no se muestre hasta que se presione el botón. Al hacer click, el método calcular() crea una instancia de la clase Circulo con el radio ingresado y llama al método calcularPerimetro(), guardando el resultado.

En el componente de Triangulo se siguió la misma lógica, pero con tres variables para los lados del triángulo. El método calcular() crea una instancia de TrianguloEscaleno con los tres valores y obtiene el perímetro de la misma manera.

TS de Círculo

TS figuras.ts | TS circulo.component.ts | TS triangulo.component.ts

src > app > components > circulo > TS circulo.component.ts > CirculoComponent

```
1  import { Component, OnInit } from '@angular/core';
2  import { CommonModule } from '@angular/common';
3  import { FormsModule } from '@angular/forms';
4  import { IonCardContent, IonInput, IonCard, IonImg, IonLabel, IonItem, IonCardTitle, IonCardHeader, IonCardSubtitle, IonButton } from "@ionic/angular/standalone";
5  import { Circulo } from '../../models/figuras';
6
7  @Component({
8    selector: 'app-circulo',
9    templateUrl: './circulo.component.html',
10   styleUrls: ['./circulo.component.scss'],
11   standalone: true,
12   imports: [CommonModule, FormsModule, IonCard, IonCardHeader, IonCardTitle, IonCardSubtitle, IonCardContent, IonItem, IonLabel, IonInput, IonButton, IonImg]
13 })
14
15 export class CirculoComponent {
16
17   radio: number = 0;
18   resultado: number | null = null;
19
20   calcular() {
21     const circulo = new Circulo(this.radio);
22     this.resultado = circulo.calcularPerimetro();
23   }
24 }
25
```

TS de Triángulo

```
figuras.ts   circulo.component.ts   triangulo.component.ts ●
src > app > components > triangulo > triangulo.component.ts > ...
import { Component, OnInit } from '@angular/core';
import { CommonModule } from '@angular/common';
import { FormsModule } from '@angular/forms';
import {
  IonCard, IonCardHeader, IonCardTitle, IonCardSubtitle,
  IonCardContent, IonItem, IonLabel, IonInput, IonButton, IonImg
} from '@ionic/angular/standalone';
import { TrianguloEscaleno } from '../../models/figuras';

@Component({
  selector: 'app-triangulo',
  templateUrl: './triangulo.component.html',
  styleUrls: ['./triangulo.component.scss'],
  standalone: true,
  imports: [CommonModule, FormsModule, IonCard, IonCardHeader, IonCardTitle, IonCardSubtitle, IonCardContent, IonItem, IonLabel, IonInput, IonButton, IonImg]
})

export class TrianguloComponent {
  ladoA: number = 0;
  ladoB: number = 0;
  ladoC: number = 0;
  resultado: number | null = null;

  calcular() {
    const triangulo = new TrianguloEscaleno(this.ladoA, this.ladoB, this.ladoC);
    this.resultado = triangulo.calcularPerimetro();
  }
}
```

Integración de los componentes en el “Home”

Para integrar los componentes en la página principal se modificaron los archivos referentes al home.page (html, ts y css)

En el HTML se agregó un ion-select con dos opciones, una para el círculo y otra para el triángulo. Con [(ngModel)] el valor que el usuario elige queda guardado en la variable figuraSeleccionada. Luego, usando *ngIf en cada componente, se muestra solo el que corresponde según la selección.

```
circulo.component.html   triangulo.component.html   home.page.html ×   circulo.component.
src > app > home > home.page.html > ion-content
Go to component
<ion-header>
  <ion-toolbar>
    <ion-title>Figura Geométrica</ion-title>
  </ion-toolbar>
</ion-header>

<ion-content>
  <div class="select-container">
    <ion-select
      placeholder="Escoja una figura geométrica..."
      [(ngModel)]="figuraSeleccionada">
      <ion-select-option value="circulo">Círculo</ion-select-option>
      <ion-select-option value="triangulo">Triángulo</ion-select-option>
    </ion-select>
  </div>

  <app-circulo *ngIf="figuraSeleccionada === 'circulo'"></app-circulo>
  <app-triangulo *ngIf="figuraSeleccionada === 'triangulo'"></app-triangulo>
</ion-content>
```

En el TypeScript se importaron ambos componentes y los módulos de Ionic necesarios para que todo funcionara correctamente dentro de un componente Standalone.

```
figuras.ts   circulo.component.ts   triangulo.component.ts   home.page.ts
src > app > home > home.page.ts > HomePage
import { Component } from '@angular/core';
import { CommonModule } from '@angular/common';
import { FormsModule } from '@angular/forms';
import { IonHeader, IonToolbar, IonTitle, IonContent, IonSelect, IonSelectOption} from '@ionic/angular/standalone';
import { CirculoComponent } from '../components/circulo/circulo.component';
import { TrianguloComponent } from '../components/triangulo/triangulo.component';

@Component({
  selector: 'app-home',
  templateUrl: 'home.page.html',
  styleUrls: ['home.page.scss'],
  standalone: true,
  imports: [CommonModule, FormsModule, IonHeader, IonToolbar, IonTitle, IonContent, IonSelect, IonSelectOption, CirculoComponent, TrianguloComponent]
})

export class HomePage {
  figuraSeleccionada: string = '';
}
```

CSS

Para los estilos se utilizó SCSS tanto en el archivo global global.scss como en los archivos individuales de cada componente. En el global se definieron estilos generales para ion-card, ion-button y ion-img. En cada componente se agregó una clase .resultado para mostrar el perímetro calculado en texto blanco y en negrita. En el home.page.scss se configuró el fondo blanco del contenido y el estilo del selector para que se vea centrado en la parte superior de la pantalla.

global.scss

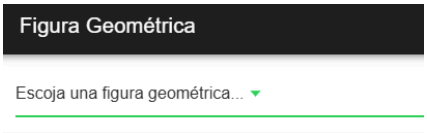
```
global.scss > .select-container
ion-content {
  --background: #ffffff;
}

.select-container {
  display: flex;
  justify-content: center;
  padding: 16px;
  border-bottom: 1px solid #ddd;
}

ion-select {
  width: 100%;
  font-size: 1rem;
  color: #333;
  --placeholder-color: #333;
  --placeholder-opacity: 1;
}

ion-card {
  border-radius: 16px;
  box-shadow: 0 4px 12px rgba(0, 0, 0, 0.1);
  margin: 16px;
}

ion-card-title {
  font-size: 1.4rem;
  font-weight: bold;
}

ion-card-subtitle {
  font-size: 1rem;
  color: #666;
}

ion-img {
  width: 200px;
  height: 200px;
  object-fit: contain;
  margin: 16px auto;
  display: block;
}

ion-button {
  margin-top: 16px;
```

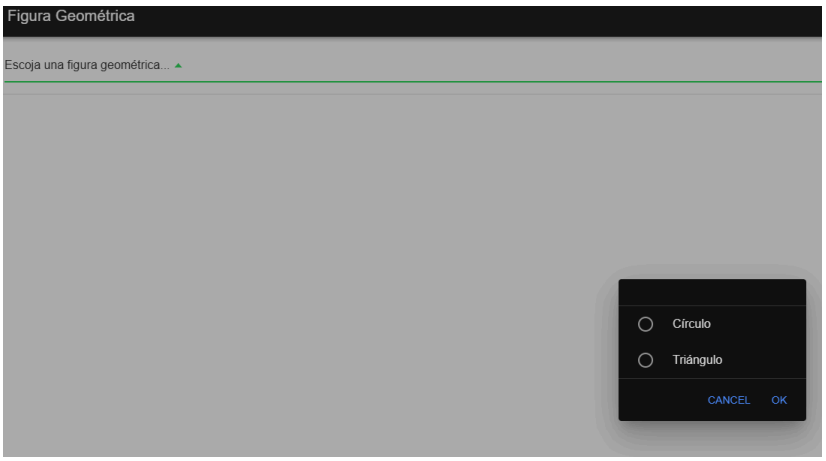
circulo y triangulo css

```css
.resultado {
  color: #ffffff;
  font-size: 1.1rem;
  font-weight: bold;
  text-align: center;
  margin-top: 16px;
}
```

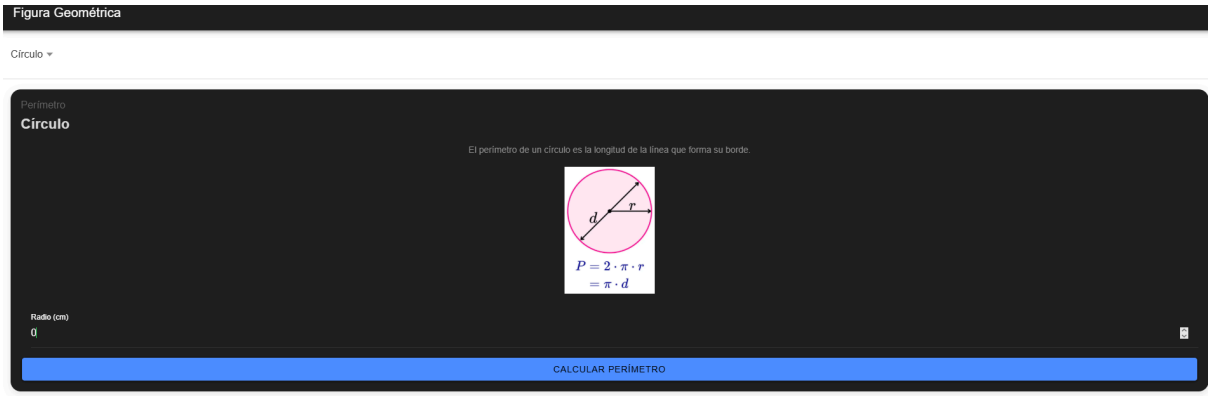
Resultado

Home page

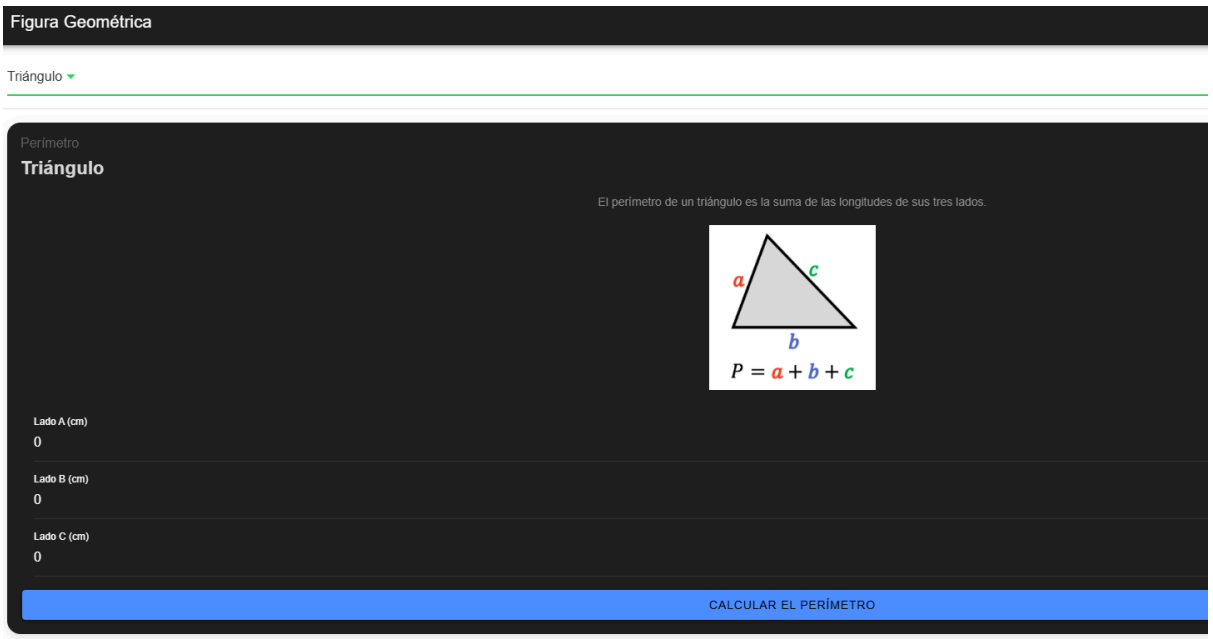


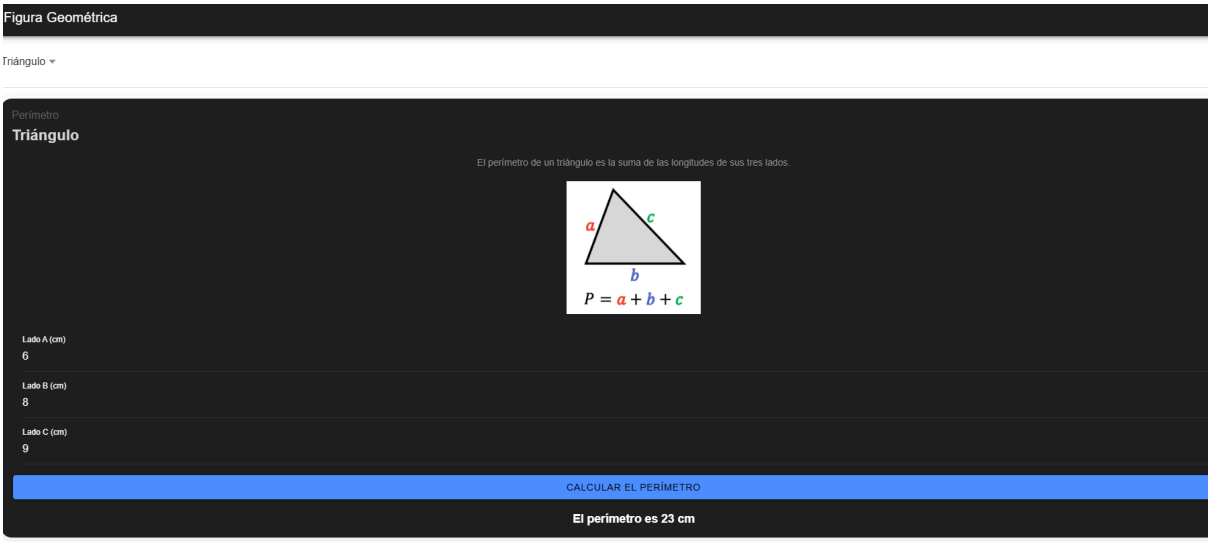


Selección Círculo



Selección Triángulo





Dificultades encontradas

1. Al principio me costó entender la diferencia entre componentes Standalone y los que usan NgModules. Esto generó algunos errores al principio porque faltaban imports que no eran evidentes.
2. También hubo un problema donde la aplicación no mostraba ningún contenido a pesar de que el código parecía correcto. El error resultó ser simplemente que el archivo home.page.html no se había guardado, lo que me tomó tiempo en identificar porque no aparecía ningún error en la consola ni en la terminal.
3. Otro inconveniente fue con las imágenes, que por defecto se mostraban ocupando toda la pantalla. Fue necesario ajustar el CSS varias veces hasta encontrar el tamaño adecuado. Del mismo modo, al principio el ion-select y el título ion-title se encimaban en el header, lo que se solucionó moviendo el select al ion-content y ajustando los estilos desde el SCSS.

Reflexión final

Este taller fue útil para entender cómo funciona Ionic con Angular en la práctica. Aprendí a crear componentes Standalone y a aplicar herencia en TypeScript siguiendo un diagrama UML. Para mejorar, necesito practicar más el manejo de imports en componentes Standalone y profundizar en SCSS para no depender tanto del ensayo y error al momento de diseñar.